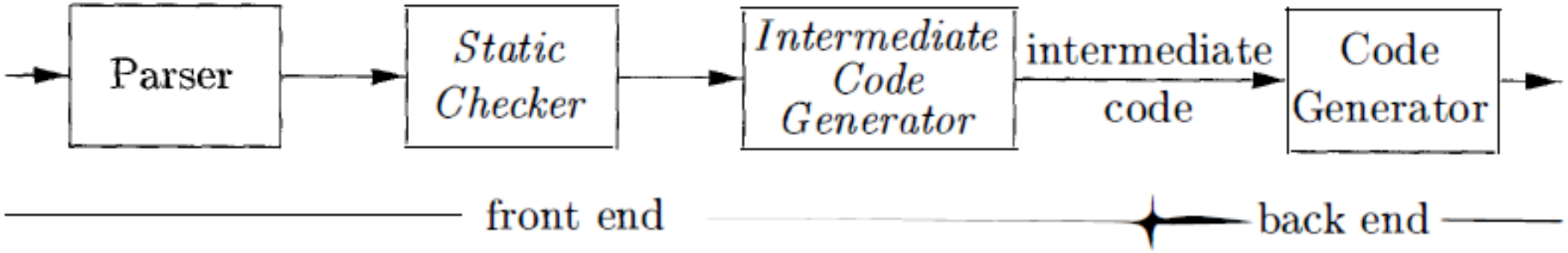
Intermediate- Code Generation

Acknowledgement

- Alfred V Aho, Monica S. Lam, Ravi Sethi, Jeffrey D Ullman- “Compilers- Principles, Techniques and Tools”

Intermediate- Code Generation

- In the analysis-synthesis model of a compiler, the front end analyzes a source program and creates an intermediate representation, from which the back end generates target code.



- Static checking includes type checking, which ensures that operators are applied to compatible operands.
- Intermediate representations include syntax trees and three-address code

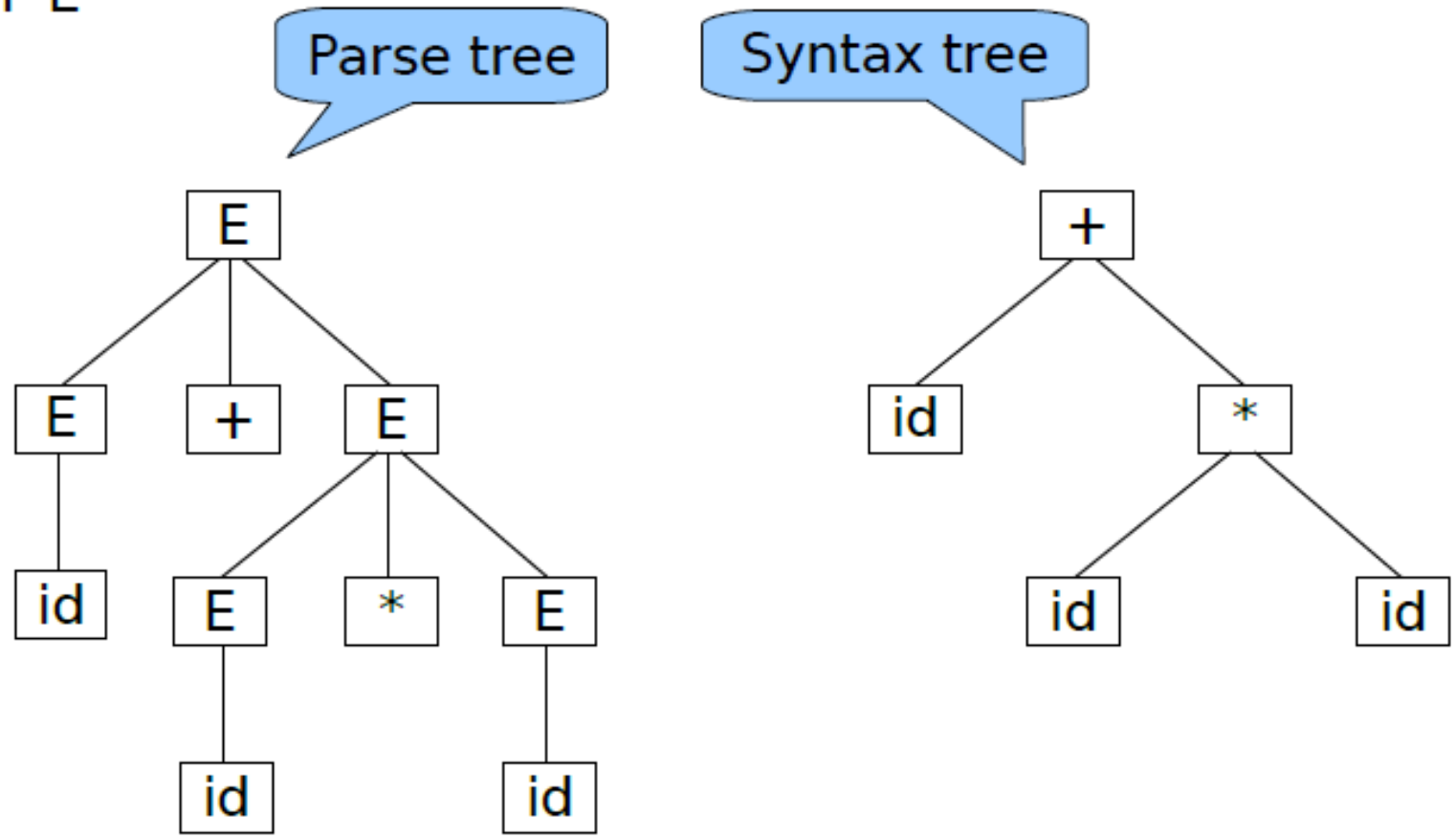
Variants of Syntax Trees

- Nodes in a syntax tree represent constructs in the source program; the children of a node represent the meaningful components of a construct.
- Parse tree: interior nodes are non-terminals, leaves are terminals
- Syntax tree: interior nodes are “operators”, leaves are operands
- Parse tree: rarely constructed as a data structure
- Syntax tree: when representing a program in a tree structure usually use a syntax tree
- Parse tree: Represents the concrete syntax of a program
- Syntax tree: Represents the abstract syntax of a program (the semantics)
- A syntax tree is often called abstract syntax tree or AST

Variants of Syntax Trees

Grammar:
E → E * E
| E + E
| id

Program: a + b * c

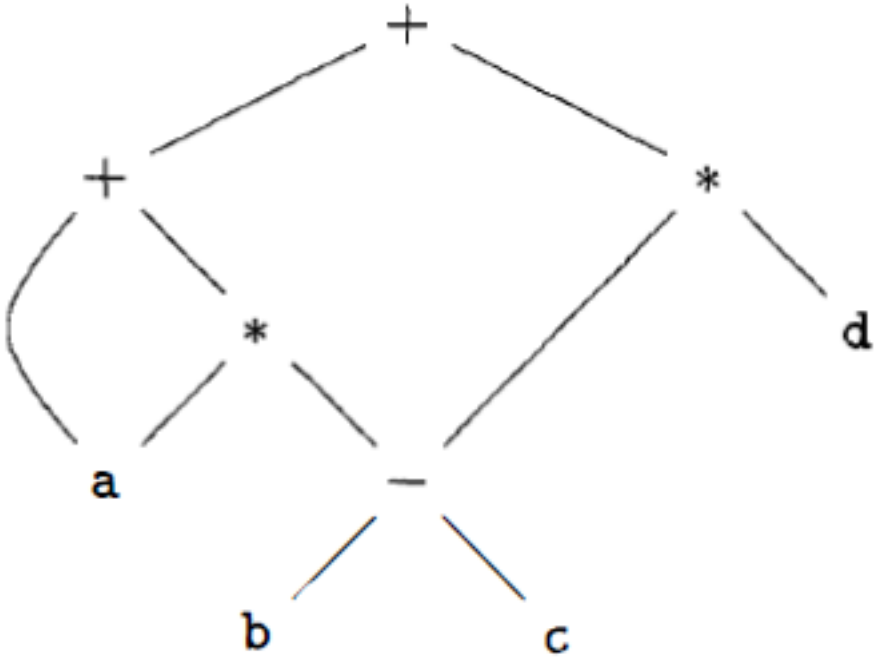


Directed Acyclic Graphs for Expressions

- DAG has leaves corresponding to atomic operands and interior nodes corresponding to operators.
- A node N in a DAG has more than one parent if N represents a common subexpression; in a syntax tree, the tree for the common sub expression would be replicated as many times as the sub expression appears in the original expression.

Directed Acyclic Graphs for Expressions

- DAG for the expression `a + a * (b - c) + (b - c) * d`



Directed Acyclic Graphs for Expressions

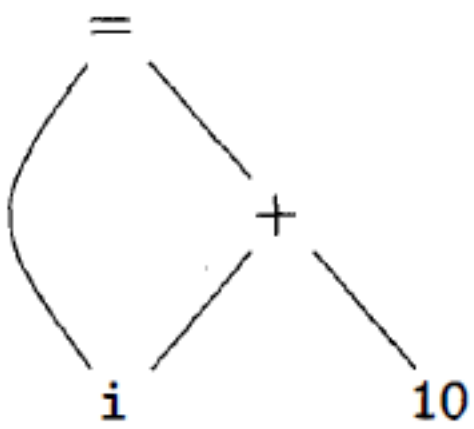
- Syntax Directed Definition to produce DAG and Steps

	PRODUCTION	SEMANTIC RULES
1)	$E \rightarrow E_1 + T$	$E.node = \textbf{new}\ Node('+', E_1.node, T.node)$
2)	$E \rightarrow E_1 - T$	$E.node = \textbf{new}\ Node('-', E_1.node, T.node)$
3)	$E \rightarrow T$	$E.node = T.node$
4)	$T \rightarrow (\ E\ )$	$T.node = E.node$
5)	$T \rightarrow \textbf{id}$	$T.node = \textbf{new}\ Leaf(\textbf{id}, \textbf{id}.entry)$
6)	$T \rightarrow \textbf{num}$	$T.node = \textbf{new}\ Leaf(\textbf{num}, \textbf{num}.val)$

1) $p_1 = Leaf(\textbf{id}, entry\text{-}a)$
2) $p_2 = Leaf(\textbf{id}, entry\text{-}a) = p_1$
3) $p_3 = Leaf(\textbf{id}, entry\text{-}b)$
4) $p_4 = Leaf(\textbf{id}, entry\text{-}c)$
5) $p_5 = Node('-', p_3, p_4)$
6) $p_6 = Node('*', p_1, p_5)$
7) $p_7 = Node('+', p_1, p_6)$
8) $p_8 = Leaf(\textbf{id}, entry\text{-}b) = p_3$
9) $p_9 = Leaf(\textbf{id}, entry\text{-}c) = p_4$
10) $p_{10} = Node('-', p_3, p_4) = p_5$
11) $p_{11} = Leaf(\textbf{id}, entry\text{-}d)$
12) $p_{12} = Node('*', p_5, p_{11})$
13) $p_{13} = Node('+', p_7, p_{12})$

The Value-Number Method for Constructing DAG's

- The nodes of a syntax tree or DAG are stored in an array of records



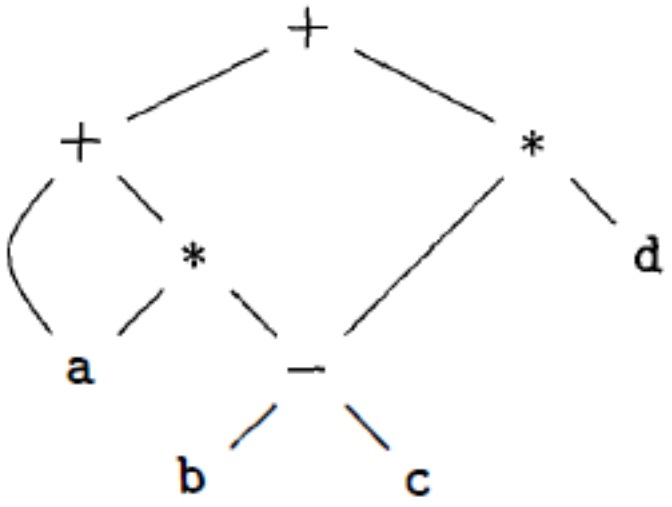
(a) DAG

1	**id**			→ to entry for **i**
2	**num**	10		
3	+	1	2	
4	=	1	3	
5	...			

(b) Array.

Three- Address Code

- In three-address code, there is at most one operator on the right side of an instruction
- Three-address code is a linearized representation of a syntax tree or a DAG



(a) DAG

$$
\begin{aligned}
t_1 &= b - c \\
t_2 &= a * t_1 \\
t_3 &= a + t_2 \\
t_4 &= t_1 * d \\
t_5 &= t_3 + t_4
\end{aligned}
$$

(b) Three-address code

Addresses and Instructions

- In Three-address code, address can be A name, A constant, A compiler-generated temporary
- Common three-address instruction forms
  - Assignment statements: *x* **:=** *y op z*, *x* **:=** *op y*
  - Copy statements: *x* **:=** *y*
  - Unconditional jumps: **goto** *lab*
  - Conditional jumps: if x goto L and if False x goto L
  - Conditional jumps: **if** *x relop y* **goto** *lab*
  - Function calls: **param** *x…* **call** *p, n…* **return** *y*
  - Indexed assignments: *x* **:=** *y*[*i*], *x*[*i*] **:=** *y*
  - Pointer assignments: *x* **:= &***y*, *x* **:= ***y*, *****x* **:=** *y*

Addresses and Instructions

```
do i = i+1; while (a[i] < v);
```

```
L:  t1 = i + 1
    i = t1
    t2 = i * 8
    t3 = a [ t2 ]
    if t3 < v goto L
```

(a) Symbolic labels.

```
100:  t1 = i + 1
101:  i = t1
102:  t2 = i * 8
103:  t3 = a [ t2 ]
104:  if t3 < v goto 100
```

(b) Position numbers.

Quadruples

- Three-address instructions are represented as "quadruples," "triples," and "indirect triples”
- A quadruple (or just " quaff') has four fields, which we call op, arg1 ' arg2 , and result.
- Three-address code for the assignment a = b * - c + b * - c

```
t1 = minus c
t2 = b * t1
t3 = minus c
t4 = b * t3
t5 = t2 + t4
 a = t5
```

(a) Three-address code

	op	arg_1	arg_2	*result*
0	minus	c		t_1
1	*	b	t_1	t_2
2	minus	c		t_3
3	*	b	t_3	t_4
4	+	t_2	t_4	t_5
5	=	t_5		a
	...			

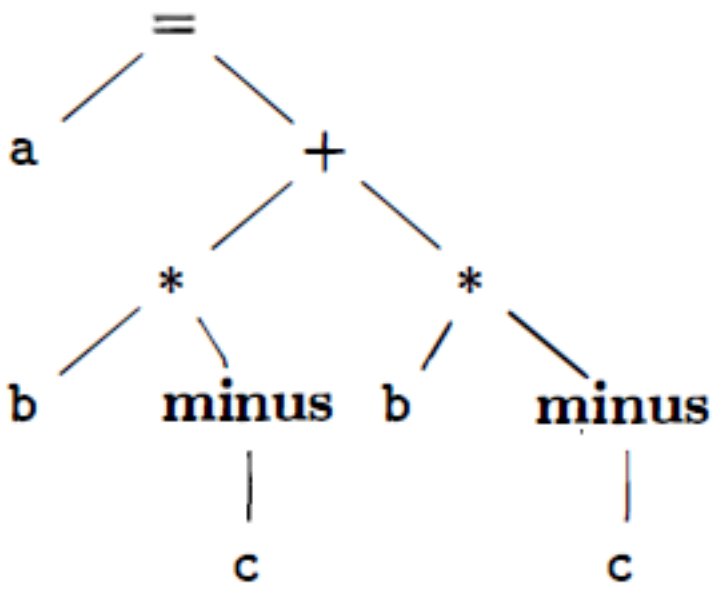
(b) Quadruples

Quadruples

- The following are some exceptions to this rule:
  - Instructions with unary operators like x = minus y or x = y do not use arg2 . Note that for a copy statement like x = y, op is =, while for most other operations, the assignment operator is implied.
  - Operators like param use neither arg2 nor result.
  - Conditional and unconditional jumps put the target label in result.

Triples

- A triple has only three fields, which we call op, arg1 , and arg2
- we refer to the result of an operation x op y by its position
- Three-Address Code for a + a * (b - c) + (b - c) * d



(a) Syntax tree

	op	arg_1	arg_2
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)
	...		

(b) Triples

Quadruples Vs Triples

- With quadruples, if we move an instruction that computes a temporary t, then the instructions that use t require no change.
- With triples, the result of an operation is referred to by its position, so moving an instruction may require us to change all references to that result .
- Indirect triples consist of a listing of pointers to triples, rather than a listing of triples themselves.

Indirect triples

- Triple and Indirect triple representation

	op	arg_1	arg_2
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)
	...		

(b) Triples

	instruction
35	(0)
36	(1)
37	(2)
38	(3)
39	(4)
40	(5)
	...

	op	arg_1	arg_2
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)
	...		

Static Single-Assignment Form

- Static single-assignment form (SSA) is an intermediate representation that facilitates certain code optimizations
- All assignments in SSA are to variables with distinct names; hence the term static single-assignment

```
p = a + b
q = p - c
p = q * d
p = e - p
q = p + q
```

(a) Three-address code.

$$p_1 = a + b$$
$$q_1 = p_1 - c$$
$$p_2 = q_1 * d$$
$$p_3 = e - p_2$$
$$q_2 = p_3 + q_1$$

(b) Static single-assignment form.

Control Flow

- Translation of statements, such as if-else-statements and while-statements, are tied to the translation of boolean expressions
- In programming languages, boolean expressions are often used to
  - Alter the flow of control: Boolean expressions are used as conditional expressions in statements that alter the flow of control
  - Compute logical values: A boolean expression can represent true or false as values

Boolean Expressions

- Grammar for Boolean Expressions

$$B \rightarrow B \mid\mid B \mid B \,\&\&\, B \mid !\,B \mid (\,B\,) \mid E \;\mathbf{rel}\; E \mid \mathbf{true} \mid \mathbf{false}$$

- the attribute rel. op indicate comparison operators <, <=, =, ! =, >, or >=

Short-Circuit Code

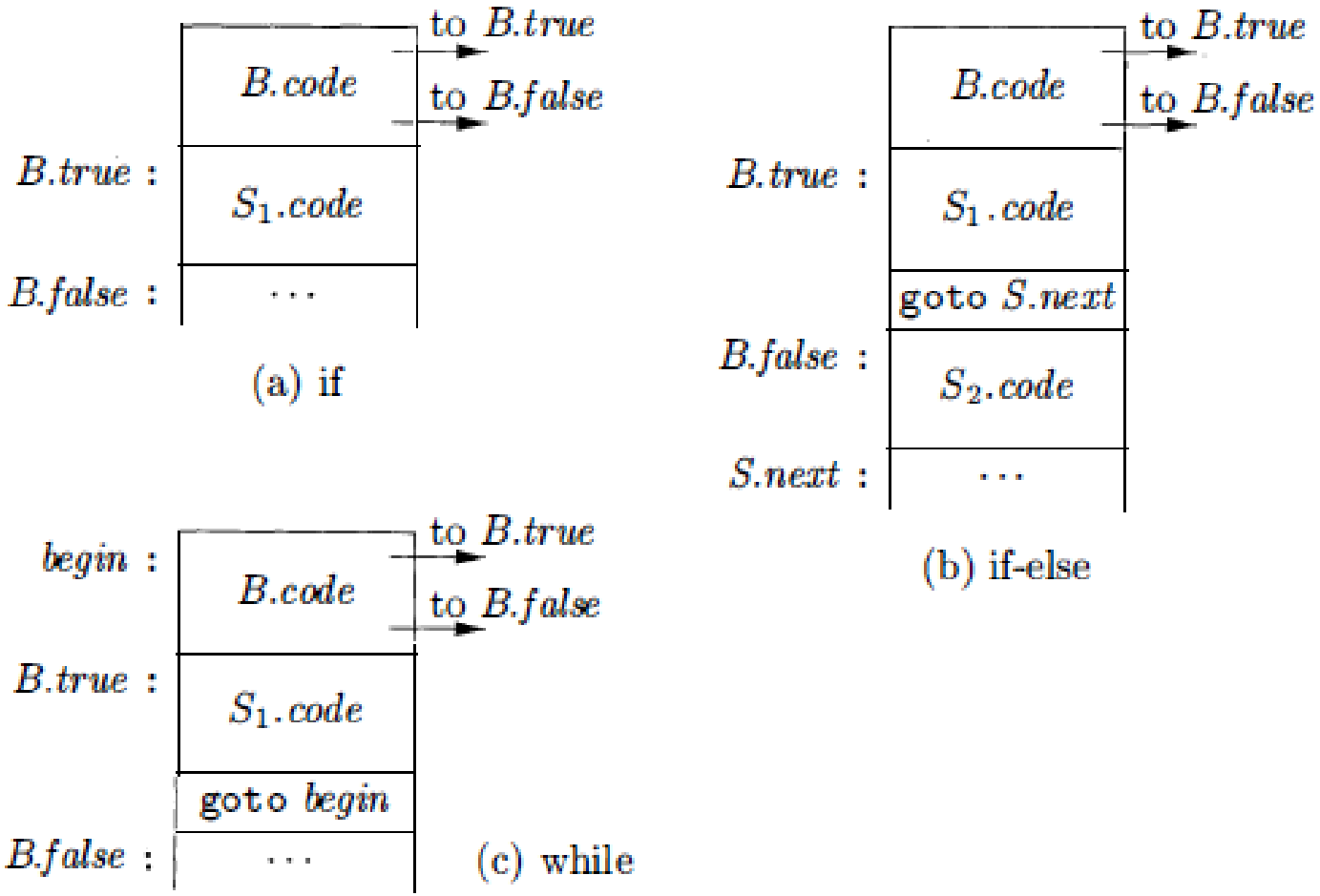
- In short-circuit (or jumping) code, the boolean operators &&, II, and ! Translate into jumps

```
if ( x < 100 || x > 200 && x != y ) x = 0;
```

```
        if x < 100 goto L2
        ifFalse x > 200 goto L1
        ifFalse x != y goto L1
L2:     x = 0
L1:
```

Flow-of-Control Statements

$$
\begin{array}{lcl}
S & \rightarrow & \textbf{if} \ (\ B \ ) \ S_1 \\
S & \rightarrow & \textbf{if} \ (\ B \ ) \ S_1 \ \textbf{else} \ S_2 \\
S & \rightarrow & \textbf{while} \ (\ B \ ) \ S_1
\end{array}
$$



Control-Flow Translation of Boolean Expressions

```
if( x < 100 || x > 200 && x != y ) x = 0;
```

```
        if x < 100 goto L2
        goto L3
L3:     if x > 200 goto L4
        goto L1
L4:     if x != y goto L2
        goto L1
L2:     x = 0
L1:
```

Backpatching

- A key problem when generating code for boolean expressions and flow-of-control statements is that of matching a jump instruction with the target of the jump.
- Backpatching
  - when a jump is generated, the target of the jump is temporarily left unspecified.
  - Each such jump is put on a list of jumps whose labels are to be filled in when the proper label can be determined.
  - All of the jumps on a list have the same target label

One-Pass Code Generation Using Backpatching

- Backpatching can be used to generate code for boolean expressions and flow-of-control statements in one pass
- Three Functions:
  - *makelist*(i) creates a new list containing three-address location i, returns a pointer to the list
  - *merge*(p_1, p_2) concatenates lists pointed to by p_1 and p_2, returns a pointer to the concatenates list
  - *backpatch*(p, i) inserts i as the target label for each of the statements in the list pointed to by p